

# Visold (VSD) Whitepaper

A plain-language technical guide to the Visold blockchain: what it does, how it reaches agreement, how its money works, how it is secured, and where it stands today.

Document version 1.0, draft for review. Date: October 8, 2026. Software version: 11.0.0.0. Chain ID: vsd-mainnet-1. Source: the code, configuration and project notes in the archive Visold\_12.zip, which holds 233 files.

How to read this paper. Sections 1 to 3 explain the project without assuming technical background. Sections 4 to 17 describe the system in depth, from cryptography to the network protocol and the command interfaces. Sections 18 to 21 cover testing, security, status and disclaimers. The appendices hold reference tables and a glossary. Figures come from the code and its configuration unless a passage labels them as estimates. Where the code and the project's own notes disagree, this paper follows the code and says so.

## Contents

| Section | Topic                                       |
|---------|---------------------------------------------|
| 1       | Summary                                     |
| 2       | Purpose and design goals                    |
| 3       | Core concepts in plain language             |
| 4       | System architecture                         |
| 5       | Cryptography and keys                       |
| 6       | Transactions, blocks and state              |
| 7       | Consensus: how the network agrees           |
| 8       | Economics and monetary policy               |
| 9       | Smart contracts: the Visold Virtual Machine |
| 10      | Layer 2 rollup                              |
| 11      | Networking                                  |

| Section | Topic                                      |
|---------|--------------------------------------------|
| 12      | Storage, pruning and fast sync             |
| 13      | Mempool and MEV protection                 |
| 14      | Self-healing, resilience and monitoring    |
| 15      | Governance and upgrades                    |
| 16      | Wallets, names and roles                   |
| 17      | Interfaces: command line, terminal and RPC |
| 18      | Testing and verification                   |
| 19      | Security model                             |
| 20      | Status, roadmap and known limitations      |
| 21      | Disclaimer                                 |
| A       | Parameter reference                        |
| B       | Glossary                                   |
| C       | Where to find each topic in the code       |

## 1. Summary

Visold, with the ticker VSD, is a cryptocurrency network and smart contract platform. The software is written in Python and organized as a modular package of 117 modules and roughly 58,000 lines of code. The project says it is meant to run on ordinary hardware, including Android phones through the Pydroid 3 app, so that individuals can operate a node without specialist equipment. A node is a running copy of the software that stores the ledger and checks every new block against the rules.

The network reaches agreement in two stages. Miners compete to solve a proof-of-work puzzle, and the winner proposes the next block. Validators, who lock up VSD as stake, then vote on that block. When validators holding at least 66.67 percent of the total stake approve a block, it is finalized. Until enough validators are registered, finality rests on proof of work alone, which means waiting for six confirmations in normal operation and twenty during bootstrap.

Visold includes its own virtual machine for smart contracts, the Visold Virtual Machine, or VVM. It has 201 instructions, eight private registers alongside the usual stack, typed values, built-in time-lock instructions, and native payment channels. A static analyzer checks contract code for risky patterns before deployment. The platform also contains a layer 2 rollup for off-chain transfers, an encrypted peer-to-peer network with NAT traversal, storage pruning with snapshot-based fast sync, and a self-healing monitor that flags suspicious activity and can freeze accounts or propose rollbacks.

Some key figures frame the rest of the paper. The target block time is 60 seconds. The initial block reward is 10 VSD and falls by 1 percent every 20,000 blocks until it reaches a floor of 0.1 VSD. One VSD equals 100,000,000 satoshi, the smallest unit. A transfer carries a fee of 1 percent of its amount. The default peer-to-peer port is 8338, and the local RPC interface listens on the next port, 8339.

Four points deserve attention before anything else. First, the supply of VSD has no cap. The block reward reaches its floor after roughly 17.5 years at the target block time, and from then on the chain issues 0.1 VSD per block indefinitely, about 52,560 VSD per year. Second, the two-thirds rule is strict. A validator set of exactly three equal-stake validators must be unanimous to finalize a block. Third, several protections exist in the code but are off by default or not yet configured: MEV protection, automatic slashing for hashrate defection, and a validity-proof backend for layer 2. Fourth, the archive contains internal audit and bug-fix reports but no independent third-party security audit.

## 2. Purpose and design goals

The archive contains no formal mission statement, so this section describes the goals that the design makes visible. It is a reading of intent drawn from the code and its notes, not an official position of the project.

The first goal is participation on modest hardware. Several features keep the storage and bandwidth needed to run a node small. Full transaction data is kept only for the most recent 600 blocks, older blocks are reduced to headers, and new nodes can start from a compressed state snapshot rather than replaying the whole chain. Consumer Android devices are named as a target platform.

The second goal is integrity over speed. Much of the code exists to make results deterministic, so that every honest node reaches the same answer from the same data. Money is handled in whole satoshi using integer arithmetic. Consensus checks reject non-finite numbers such as NaN and infinity. Block validation never depends on local conditions such as how full a node's mempool happens to be.

The third goal is safe programmability. Smart contracts run in a sandbox with no access to the operating system or the network. A static analyzer looks for patterns such as reentrancy, misuse of payment channels, and time-lock mistakes. Typed values and dedicated time-lock instructions make a contract's intent easier to read and check.

The fourth goal is fairer transaction ordering. The optional commit-reveal scheme hides transaction details until after they have been committed, which reduces the opportunity for outside bots to front-run a transaction.

The fifth goal is resilience. The node includes a circuit breaker that can halt risky operations, invariant checks that test whether money is conserved, a sentinel that watches a primary node, and a self-healing layer that responds to anomalies in graded steps.

The sixth goal is controlled change. Protocol upgrades pass through signaling thresholds, activation heights and a rollback mechanism, so that changes can be coordinated across independent operators.

### 3. Core concepts in plain language

This section defines the building blocks used throughout the paper. Readers familiar with other blockchains will recognize most of them, but Visold's specifics are noted where they differ.

**Coins and units.** VSD is the native coin. The ledger records every amount as a whole number of satoshi, where 1 VSD equals 100,000,000 satoshi. Working in whole numbers avoids the rounding drift that decimal fractions can cause in software, so every node computes identical balances.

**Accounts and addresses.** An address is the identifier that receives and sends coins. Visold addresses begin with the letters VSD, followed by a Base58 encoding of 20 bytes derived from the owner's public key. The public key comes from a private key that only the owner should hold. Whoever controls the private key controls the coins at that address.

**Wallets and keys.** A wallet holds keys and signs transactions on the owner's behalf. A wallet can be created together with a 24-word backup phrase, from which the keys can be regenerated. Anyone who learns the phrase can take the funds, so it should be written down, stored offline and never shared.

**Transactions.** A transaction is a signed instruction. It can transfer VSD, deploy a smart contract, call one, register a role or a name, or carry layer 2 data. The signature proves that the holder of the private key authorized the action.

**Blocks and the chain.** A block is a batch of transactions with a header. The header contains the hash of the previous block, so each block commits to the entire history before it. Changing an old block would mean recomputing every block after it. The chain is the ordered sequence of blocks from the genesis block onward.

**Mining.** Miners search for a nonce that makes a block's hash fall below a target. The difficulty system adjusts that target so that blocks arrive about once a minute on average. The miner who finds a valid block proposes it to the network.

**Validators and stake.** A validator locks up VSD as stake and casts votes on blocks. Votes are weighted by stake, so a validator with more stake carries more weight. Signing two conflicting blocks at the same height is an offence that can lead to slashing, which removes part of the validator's stake.

**Finality.** A block is final when the protocol treats it as effectively irreversible. Visold offers two routes. Proof-of-work finality rests on how many blocks have been built on top of a block. Validator finality rests on a supermajority of stake approving the block.

**Nodes and peers.** A node runs the software, keeps a copy of the chain and connects to other nodes, called peers. Peers exchange blocks, transactions and status messages. Every node checks everything it receives, so it does not need to trust the peer that sent the data.

**The mempool.** The mempool is a node's waiting area for transactions that have been announced but not yet included in a block. Block builders select from it, usually preferring higher fees.

**Smart contracts.** A smart contract is code stored on the chain that runs when someone calls it. Visold contracts run on the VVM described in Section 9.

**Layer 2.** A layer 2 system processes transactions outside the main chain and periodically posts compact commitments back to it. Visold's layer 2 is a rollup, described in Section 10.

## 4. System architecture

Visold began as a single source file of 55,370 lines. The project then reorganized that file into a package named visold. The package has 117 modules grouped into 22 bounded contexts. A bounded context is a group of code responsible for one area, such as cryptography, storage, networking or the virtual machine. Keeping each area separate makes the software easier to test, review and change without unintended side effects.

The contexts sit in twelve layers, numbered L0 to L11. A module may import code from its own context or from a lower layer, but never from a higher one. This rule rules out import cycles, which are circular dependencies that make software difficult to reason about. A checking tool named `check_architecture` enforces the rule and is meant to run after every change.

| Layer | Contexts                       | Role                                                                                                                           |
|-------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| L0    | kernel                         | Configuration, logging, clock, metrics, units, message identifiers and shared helpers. Depends on nothing else in the package. |
| L1    | crypto, economics, governance  | Cryptographic primitives, economic monitoring and reward rules, and upgrade governance.                                        |
| L2    | consensus, rollup, selfhealing | Consensus rules that do not need chain state, the layer 2 core, and the self-healing system.                                   |
| L3    | resilience, vm, wallet         | Circuit breaker and safety checks, the virtual machine, and wallet key handling.                                               |
| L4    | ledger                         | Transaction and block structures.                                                                                              |
| L5    | state, storage                 | The single writer that applies state changes, and persistence.                                                                 |
| L6    | mempool                        | The pending transaction pool and optional MEV protection.                                                                      |
| L7    | chain                          | Blockchain management, reorganization, role manager, consensus engine and layer 2 sequencer.                                   |
| L8    | network                        | Peer-to-peer networking, NAT traversal, discovery, TLS, reputation and light clients.                                          |
| L9    | api, identity, mining, testing | The JSON-RPC server, the name service, miners and test harnesses.                                                              |
| L10   | node                           | The node itself, user accounts and the security gate.                                                                          |
| L11   | cli                            | The terminal interface, command-line handler and program entry point.                                                          |

The largest modules are the peer-to-peer network class, the blockchain class and the storage class, each exceeding 5,000 lines. The modular split was checked for behavioral equivalence against the original file, and Section 18 summarizes those checks.

The program starts from the file `visold_vsd.py`, which serves as the entry point and keeps older flat names working for existing scripts. Running it with no options starts the interactive node. The test option runs the embedded test suite, and the cli option opens the command-line wallet and node tools. The project's README says the program can run on Android by opening `visold_vsd.py` in Pydroid 3 and pressing Run, provided the visold folder sits beside it.

## 5. Cryptography and keys

Visold builds on established cryptographic primitives. The implementation is written in Python, with an optional faster backend for signatures.

**Signatures.** Transactions are signed with ECDSA on the `secp256k1` curve, the same curve used by Bitcoin and many other systems. The signing digest is SHA-256 applied twice, which is the historical behavior that existing signatures on the network rely on. The software uses the cryptography library by default. When the optional `libsecp256k1` backend is installed and passes a self-test at startup, the wallet uses that instead. The October 2026 acceleration patch keeps private-key objects inside each wallet so they are not rebuilt for every signature, and it allows signature checks to run in parallel worker processes.

**Hashing.** SHA-256 produces block hashes, transaction identifiers and the nodes of Merkle trees. Double SHA-256 appears in address derivation and in the signing digest.

**Addresses.** To derive an address, the software hashes the public key with SHA-256, hashes the result again, keeps the first 20 bytes and encodes them in Base58 with the prefix VSD. Contract accounts use the same format.

**Keystore.** Private keys saved to disk are encrypted with AES-256 in CBC mode. The encryption key is derived from the user's password with Argon2id, a memory-hard function, when the `argon2-cffi` package is installed. Without that package, the software falls back to PBKDF2-HMAC-SHA256 with 260,000 iterations. Argon2id is more resistant to attack with specialized hardware, so installing it is the stronger choice.

**Backup phrase.** The wallet can express a 256-bit private key as 24 words, following the general design of BIP-39. The code notes that its word list is padded or trimmed to exactly 2,048 entries. A Visold phrase should therefore not be assumed to restore in standard

BIP-39 wallets, and anyone relying on a phrase should test restoration with the Visold software first.

Deterministic derivation. The wallet module can derive wallet keys deterministically from a secret, so a single backup can regenerate the wallet's keys.

Verifiable random functions. The consensus code includes an elliptic-curve verifiable random function built on secp256k1 with SHA-256, following the ECVRF-secp256k1-SHA256-TAI design. Nonces are generated deterministically under RFC 6979. A VRF produces an output that anyone can check against the public key, which makes it useful for fair and unpredictable selection. Its status in block production is described in Section 7.

Transport security. Connections between peers are wrapped in TLS. Each node generates its own self-signed P-256 certificate, valid for 90 days. Peers accept a certificate the first time they see it and pin its fingerprint afterwards, a model known as trust on first use. The P-256 curve used for TLS is separate from the secp256k1 curve used for transaction signatures.

Independent review. The archive contains detailed internal audit reports covering cryptographic code paths. It contains no external cryptographic review, and this paper does not claim one.

## 6. Transactions, blocks and state

Transaction types. Five transaction types are defined. A transfer moves VSD between addresses. A deploy transaction installs a smart contract, and a call transaction runs one. A register transaction records a role registration or an unstake, and it is also used to claim names. A rollup transaction carries a batch of layer 2 activity to the main chain.

Transaction fields. A transaction records a version number, a transaction identifier, the sender and receiver addresses, the amount, the fee, a timestamp, the sender's public key and signature, an optional memo, a nonce that orders the sender's transactions, an expiry time, and fields used by the virtual machine. If no expiry is set, a transaction expires after one hour. No transaction may be set to expire more than 24 hours after creation.

Signing format. Newer transactions use a canonical encoding that marks the boundaries of each field and separates signing data from other uses. This prevents two different transactions from producing the same signing bytes by accident. Amounts and fees are committed in whole satoshi, and gas prices are committed in satoshi per unit of gas.

**Block structure.** A block consists of a header and a body. The header records the block's own hash, the hash of the previous block, the Merkle root, the state root, the timestamp, the difficulty target and the nonce. The Merkle root is a single hash that summarizes every transaction in the block, so one transaction can be proven to be included without downloading the others. The state root commits to all balances, nonces, contract data and role records after the block is applied. The body holds the transactions, starting with a coinbase transaction that pays the block reward.

**Block size.** Under consensus rules, a block may not exceed 4,096,000 bytes, which is about 3.9 MiB. Nodes that build candidate blocks choose a size between a 1 MiB floor and that cap, guided by local conditions such as the mempool backlog. That choice is a building policy only, and other nodes do not judge blocks by it, because validation uses the fixed consensus cap. The candidate size grows by 10 percent when a backlog forms and shrinks by 5 percent when it clears.

**Timestamps.** A block's timestamp must be later than the median of the previous 11 block timestamps, and it may not be more than 7,200 seconds, or two hours, ahead of the validating node's clock. Non-finite timestamps are rejected outright.

**Genesis.** The genesis block is not mined. Its timestamp is 1224720000, which corresponds to 00:00:00 UTC on October 23, 2008. The configured amount minted by the genesis transaction is zero, so the code creates no pre-mined supply at genesis.

**Account state.** The state holds account balances in satoshi, account nonces, contract code and storage, role records and stakes, and name claims. A single state engine is the only component allowed to change this data, which keeps updates in one defined order.

**Conservation.** A consistency check confirms that the sum of all balances equals cumulative issuance. Burned funds are credited to a designated burn address rather than destroyed, so they remain part of the total and the check still holds.

## 7. Consensus: how the network agrees

Visold combines several mechanisms. Proof of work creates blocks. Stake-weighted validator voting finalizes them under a Byzantine fault tolerant rule. A verifiable random function supports fair leader selection, and difficulty and hashrate rules keep block times near the target. The consensus engine brings these parts together, though some are only partly wired into block production, as noted below.

## 7.1 Proof of work

Miners search for a nonce that makes the block hash meet a target derived from the difficulty. Difficulty is expressed as a fractional number of leading zeros in the hexadecimal form of the hash, which lets it change in small steps. The chain starts at difficulty 5.15 and keeps the value between 4.0 and 63.0.

## 7.2 Difficulty adjustment

The difficulty engine uses a linearly weighted moving average over the last 20 blocks, so the most recent blocks count most. The solve time of each block, measured from its parent, is clamped between 1 and 360 seconds. That clamp limits how far a single manipulated timestamp can move the result. No single adjustment may change the difficulty by more than a factor of 1.5. The first three blocks keep the initial difficulty, which gives the chain a stable start. The aim is an average of 60 seconds per block, with a tolerance band of plus or minus 5 seconds.

## 7.3 Hashrate governor

Each miner's local software is subject to a hashrate cap. The caps are computed so that total hashing power on the network stays matched to the difficulty target. Because the caps are enforced by the local software, they depend on the operator's honesty. The governor is therefore best understood as guidance for well-behaved miners, with the consensus layer providing the enforcement backstop described next.

## 7.4 The rate-defection auditor

Every 50 blocks, the consensus engine audits miners over a rolling window of 500 blocks. For each miner holding at least 3 percent of the audited hashrate, it compares the blocks that miner actually won with the number expected from its cap. The ratio of actual to expected wins is the signal. A ratio above 2.0 counts as a flagged window, and three consecutive flagged windows would lead to slashing. The threshold is set so that an honest miner would be flagged by chance only very rarely. At the target block time, the shortest time to slash a persistent defector is about 25 hours, which is three windows of 500 blocks at 60 seconds each.

Automatic slashing is off by default. The software starts in observe-only mode. It logs the miners it would have penalized and broadcasts advisory evidence, but it does not slash anyone. The project notes call for several weeks of observation, to confirm there are no

false positives, before slashing is enabled through a coordinated configuration change and network restart. A separate offence counter halves every 10,000 blocks, so older offences fade gradually.

## 7.5 Proposer selection

The consensus engine contains a routine that selects a leader using a verifiable random function. Each candidate's VRF output is compared, and the lowest output wins, which gives a selection that is deterministic and publicly checkable. In the code reviewed for this paper, however, the routine is defined but not called from the block production path, so this paper does not describe it as active consensus behavior. The reward rules, meanwhile, pay the proposer share to the miner whose proof of work produced the block.

## 7.6 Validators and BFT finality

When at least three validators are registered, the chain enters Byzantine fault tolerant mode. Validators cast stake-weighted votes on blocks, and a block is finalized once the approving stake is at least 66.67 percent of total stake. The check uses integer arithmetic: the approving stake multiplied by one million must be at least 666,700 multiplied by the total stake. Votes travel to other nodes over the peer-to-peer network.

Before BFT mode is active, finality depends on proof-of-work depth. A block is treated as final after six confirmations in normal operation, or after twenty confirmations while fewer than three validators are registered.

Because the threshold sits just above two-thirds, small validator sets behave differently from what many people expect. The table below shows how many votes are needed for finality when all validators hold equal stake.

| Validators | Votes needed | Share of stake | Comment                                               |
|------------|--------------|----------------|-------------------------------------------------------|
| 3          | 3            | 100 percent    | Two of three is 66.67 percent, which falls just short |
| 4          | 3            | 75 percent     |                                                       |
| 5          | 4            | 80 percent     | Three of five, or 60 percent, is not enough           |
| 6          | 5            | 83.3 percent   | Four of six, or 66.67 percent, falls just short       |
| 9          | 7            | 77.8 percent   | Six of nine, or 66.67 percent, falls just short       |

The practical consequence is that with three equal-stake validators, one offline validator stops BFT finality until it returns. Operators planning a small validator set should account for this. Unequal stakes change the arithmetic, and the check always uses the exact stake amounts.

## 7.7 Slashing and double-sign evidence

If a validator signs two different blocks at the same height, the node that detects both signatures builds a double-sign evidence packet. The packet contains both signatures, both block hashes, and the validator's identity, which is bound to its public key. The packet is broadcast to peers. Every receiving node verifies the evidence independently and applies the penalty itself, so honest nodes reach the same outcome without relying on one node's report. The configured slashing rate for a malicious vote is 10 percent of stake.

## 7.8 Fork signaling

Changes that need miner agreement use signaling. Miners include the protocol version they support in the blocks they produce. If at least 75 percent of the last 100 blocks signal support for a new version, the change can lock in for a future activation height. Section 15 describes the full lifecycle.

## 7.9 Fail-closed checks

The consensus code is written to fail closed, meaning that bad inputs cause rejection rather than acceptance. Non-finite difficulty values are refused before any arithmetic happens. Malformed hashes make the proof-of-work check fail. Timestamps containing

NaN or infinity are refused, and a candidate block with a malformed difficulty fails to mine rather than producing an invalid result. Several of these checks were added after the October 2026 audit reviewed edge cases.

## 8. Economics and monetary policy

### 8.1 The base reward

Each block pays a base reward that depends only on the block's height. The reward begins at 10 VSD. For every complete 20,000 blocks, it is multiplied by 0.99, and the result is rounded half up using exact integer arithmetic, so every node computes the same value. The reward never falls below 0.1 VSD. At a 60-second target, 20,000 blocks take about 13.9 days, so the reduction happens roughly every two weeks. Those calendar figures are estimates, because real block times vary around the target.

In plain terms, the base reward at height  $h$  is the larger of 0.1 VSD and 10 VSD multiplied by 0.99 raised to the power of  $h$  divided by 20,000, rounded down to the whole number of periods.

### 8.2 Emission schedule

The table below shows the schedule. Block heights are exact. Calendar years are estimates based on a 60-second block time.

| Milestone        | Block height<br>(approximate) | Years at 60 seconds per block<br>(approximate) | Base reward<br>per block |
|------------------|-------------------------------|------------------------------------------------|--------------------------|
| Genesis          | 0                             | 0                                              | 10.00 VSD                |
| Year 1           | 525,600                       | 1                                              | 7.70 VSD                 |
| Year 2           | 1,051,200                     | 2                                              | 5.93 VSD                 |
| Year 3           | 1,576,800                     | 3                                              | 4.57 VSD                 |
| Year 5           | 2,628,000                     | 5                                              | 2.68 VSD                 |
| Year 10          | 5,256,000                     | 10                                             | 0.72 VSD                 |
| Year 15          | 7,884,000                     | 15                                             | 0.19 VSD                 |
| Floor<br>reached | about 9,180,000               | about 17.5                                     | 0.10 VSD                 |

Cumulative base issuance follows from the same schedule. Through year one, about 4.64 million VSD have been issued. Through year five the total is about 14.66 million, and through year ten it is about 18.57 million. Before the floor is reached, the total is about 19.8 million VSD. These totals cover only the base reward and exclude transaction fees, which are redistributed rather than newly issued.

After the floor, the chain issues 0.1 VSD per block indefinitely. At the target block time that is about 144 VSD per day, or about 52,560 VSD per year.

The base issuance is front-loaded. About 18.6 million of the roughly 19.8 million VSD issued before the floor arrives in the first ten years. After the floor, issuance continues at a constant absolute rate. Because that amount stays the same while the total grows, the percentage growth rate falls over time, but the total supply keeps increasing. The schedule is tied to block height rather than to the clock. If blocks arrive faster than the target, the same coins are issued in less time, and if they arrive more slowly, they are issued over a longer period.

### 8.3 How each block's reward is shared

Each block's total reward is its base reward plus the transaction fees included in the block. That total is divided as follows. The block proposer, which is the miner whose proof of work produced the block, receives 20 percent. All active miners share 45 percent in

proportion to their hashrate scores. All active validators share 35 percent in proportion to their stake. If no validators are registered, the validator share moves to the miners, who then divide 80 percent between them.

Miner activity is measured over the canonical proof-of-work blocks of the last 100 blocks, using only data that the chain itself records, so every node computes the same split. Integer division leaves a small remainder, typically between zero and two satoshi per block, and that remainder is sent to the burn address. No other burning takes place.

As a worked example, consider height 1,000,000. The base reward there is 10 VSD multiplied by 0.99 to the 50th power, which is about 6.05 VSD. Suppose the block also carries 0.50 VSD of fees, for a total of about 6.55 VSD. The proposer receives about 1.31 VSD, the active miners share about 2.95 VSD, and the validators share about 2.29 VSD. If no validators were registered, the miners would receive about 5.24 VSD in total.

## 8.4 Transaction fees

A transfer's fee is 1 percent of the amount transferred. The calculation uses integer arithmetic on satoshi, so there is no rounding ambiguity. A transfer of 250 VSD therefore carries a fee of 2.5 VSD. Identity claims and certain registration transactions pay a flat minimum of 1,000 satoshi, which is 0.00001 VSD. Node policy also requires at least that minimum for a transaction to enter the mempool, as a defense against spam. Fees are added to the block reward and distributed as described in Section 8.3, so fees are not burned.

A configuration comment describes the transfer fee as applied from the receiver's side. The fee calculation itself depends only on the amount and does not name a payer. This paper describes the calculation as the code states it. The payer rule should be confirmed in the state-transition code before publication.

## 8.5 Stake requirements and penalties

Participants register as miners or investors, as described in Section 16. Initial registration requires at least 10 VSD of stake for a miner and at least 200 VSD for an investor. The configured slashing rate for a malicious vote is 10 percent of stake. Validator votes are weighted by stake.

## 8.6 Concentration monitoring

An economic monitor watches reward distribution and validator behavior for signs of collusion or reward farming. It raises an alert when more than 40 percent of rewards go to a single address.

## 8.7 What the economics do and do not promise

This paper reports rules, not market outcomes. The design is deterministic, and its monetary rules are written in code that anyone can read. Because supply has no cap, long-run security depends on continuing fee income and tail issuance rather than on scarcity. The paper makes no forecast of price or value.

# 9. Smart contracts: the Visold Virtual Machine

## 9.1 Overview

The VVM is a deterministic machine that executes contract bytecode. The project describes it as a stack-based machine with a register file and typed values. Execution is sandboxed, so a contract cannot reach the operating system or the network. Every instruction has a gas cost, and gas limits how much work a single call may perform.

## 9.2 Stack and registers

Values are 256-bit words. Each call frame has a stack and eight registers, named R0 through R7. Registers start at zero in every new frame and are never shared between a parent call and a child call. Register instructions never touch storage or emit logs, so they remain safe inside read-only calls. RCLEAR wipes all eight registers before a sensitive computation returns, and RPUSH\_ALL and RPOP\_ALL save and restore the whole register file across nested calls. Register moves and single-register stores and loads cost 1 gas each. Register addition costs 2 gas, clearing costs 3 gas, and the bulk push and pop cost 8 gas each.

## 9.3 Typed values

Each stack value carries one of five type tags: unsigned integer, address, boolean, satoshi amount, or hash. Arithmetic results are unsigned integers by default. Instructions that read balances return satoshi-tagged values, and instructions that produce addresses return address-tagged values. Contracts can read a tag with TYPEOF, require a particular

tag with TYPEASSERT, change a tag explicitly with TYPESET, test a tag without reverting with TYPECHECK, and load a persisted tag with TYPEDLOAD. The purpose is to catch mistakes such as treating a balance as an address.

## 9.4 Time-lock instructions

Four instructions express time rules directly. AFTER height reverts if the current block is below the given height, at 8 gas. BEFORE height reverts if the current block has reached the given height, also at 8 gas. WINDOW start end reverts unless the current block lies inside the range, at 12 gas. BLOCKAGE pushes the current block height, at 2 gas. Each one replaces a multi-instruction comparison pattern that costs more gas and is easier to get wrong. The static analyzer flags zero-height guards, windows whose start is not below their end, and BEFORE guards with no matching lower bound.

## 9.5 Native payment channels

Payment channels are built into the virtual machine as native instructions, so the node enforces the channel rules itself rather than relying on each contract to implement them. CHAN\_OPEN locks a deposit from the caller and costs 5,000 gas. CHAN\_CLOSE performs a cooperative close using a signature from the counterparty and costs 8,000 gas. CHAN\_DISPUTE raises a dispute, or forces a close once the timeout has passed, at 10,000 gas for the first call and 12,000 gas for a forced close.

| Channel state | How it is reached                            | What can happen next            |
|---------------|----------------------------------------------|---------------------------------|
| Open          | CHAN_OPEN locks the deposit                  | Cooperative close, or a dispute |
| Closed        | CHAN_CLOSE with the counterparty's signature | Nothing further                 |
| Disputed      | CHAN_DISPUTE raised by one party             | Forced close after the timeout  |
| Force-closed  | CHAN_DISPUTE after the timeout has passed    | Nothing further                 |

Several rules apply throughout. Dispute timeouts are limited to between 10 and 50,000 blocks. Sequence numbers in disputes must strictly increase, which stops an old state from being replayed. Balances at close must sum to the total deposit. Duplicate open

channels between the same pair of parties are refused. Channel instructions cannot run inside read-only calls, and signatures are checked by recovering the signing key on the secp256k1 curve.

## 9.6 Calls, creation and value transfers

The VVM supports ordinary calls, call-code and delegate-call variants, static calls that cannot change state, and two contract creation instructions, CREATE and CREATE2. Value sent to a nested call is first recorded in a temporary overlay, and it becomes permanent only if the child call succeeds. If the child reverts, the overlay is discarded. Balance reads during execution include the in-flight overlay, so contract code sees the balance it would actually have. The October 2026 fixes corrected this behavior.

## 9.7 Events and blockchain context

Contracts can emit indexed events with LOG0 through LOG4. The node stores these events and makes them searchable by contract and by topic through RPC. Contracts can also read blockchain context, including the block number, timestamp, difficulty, coinbase address and chain identifier. Visold adds several chain-specific reads. These cover the staking balance, the validator count, the VSD balance, the transaction sender, and BLOCKFINALIZED, which reports whether an earlier block is final. BLOCKFINALIZED answers only for blocks strictly before the block being applied, and queries about the current or later blocks return zero. This makes the answer the same on every node during execution.

## 9.8 Static analysis

Before deployment, the static analyzer reads contract bytecode and looks for risky patterns. It checks for reentrancy, where a contract calls out to another contract and then changes its own state in an unsafe order. It checks payment-channel usage for missing exit paths, missing dispute paths and dispute patterns that do not match an open channel. It checks time-lock usage for the mistakes described in Section 9.4. The analyzer is pattern-based, so it can miss risks that fall outside its rules and can flag code that is actually safe. Developers should treat its findings as a review aid, not as proof that a contract is safe.

## 9.9 Developer tools

Developers can deploy and call contracts, read storage and receipts, list deployed contracts, and simulate a call without committing it. The gas estimation and simulation methods report expected costs and outcomes before a transaction is sent. Section 17 lists the RPC methods.

## 9.10 Assurance status

The archive includes regression tests for virtual machine fixes and detailed internal reports. It does not include a separate formal specification of the VM, and it does not include a third-party audit. A contract platform that will hold real value should undergo external review before launch.

# 10. Layer 2 rollup

## 10.1 Purpose

The layer 2 rollup lets participants move VSD between accounts on a separate ledger, which is periodically committed to the main chain. The main chain stores only compact commitments, so the design aims to raise throughput while keeping the main chain as the final record.

## 10.2 Deposits, withdrawals and transfers

When a user deposits VSD into layer 2, the coins are locked in escrow on the main chain and the same amount is credited to the user's layer 2 balance. Withdrawals reverse the process. Transfers inside layer 2 are signed messages that adjust balances and nonces on the rollup ledger. The RPC interface provides methods for reading the layer 2 balance and state root, sending a layer 2 transaction, depositing, withdrawing and triggering a manual rollup.

## 10.3 Layer 2 transaction format

A layer 2 transaction is deliberately minimal. It carries the sender, receiver, amount, nonce and signature. It has no fee field and no memo, which keeps each transaction small. A layer 2 transfer therefore has no fee field of its own.

## 10.4 State tree

Layer 2 balances are stored in a Merkle tree whose leaves are ordered by address. The root of that tree is the layer 2 state root, which the sequencer commits to the main chain. The project chose a simple sorted-address Merkle tree over more complex structures for clarity and auditability, and its design comments explain the trade-offs.

## 10.5 Sequencer and batches

A sequencer collects layer 2 transactions and seals them into batches. A batch closes when it reaches 2,048 transactions or when it is 30 seconds old, whichever happens first. The sequencer then produces a proof for the batch and posts a rollup transaction to the main chain, carrying the new state root and batch data. The node keeps a history of the 128 most recent layer 2 state roots so that recent history can be checked.

## 10.6 Proof backends and their status

A rollup is only as trustworthy as its proofs. The code defines a common proof interface with three implementations. LocalDevBackend uses HMAC-SHA256, which is a message authentication code. Its own documentation states plainly that it is not a zero-knowledge proof and not even a proof, because anyone who holds the shared key can produce a tag that verifies. SimulatedProofBackend is an older name kept for compatibility. SubprocessSNARKBackend is a wrapper that calls an external Groth16 or PLONK prover program, which would produce genuine succinct proofs once a real prover is installed and configured.

The default configuration sets the proof backend to a placeholder value that reads no-backend-configured. The code refuses to run the sequencer or verify proofs on mainnet with a backend that identifies itself as unsafe. In other words, the software guards against accidental use of the development backend on the main network. A production layer 2 still requires a real backend to be installed and tested, and Section 20 lists this as a limitation.

## 10.7 Summary

The rollup's data structures, sequencer, state management and RPC methods are implemented and covered by tests. Its security case depends on validity proofs from a real backend, which the archive does not provide. Until that backend is in place, layer 2 balances should be understood as depending on the honesty of the sequencer.

## 11. Networking

### 11.1 Transports

Nodes communicate over two transports. TCP provides ordinary reliable connections. The UDP transport adds its own reliability layer, message fragmentation, forward error correction so that lost fragments can be rebuilt, per-peer sessions and latency tracking. The node can fall back to TCP when UDP delivery does not work.

### 11.2 Encryption

All peer sockets are wrapped in TLS, and Section 5 describes the certificates and the trust-on-first-use model. Encryption protects against passive eavesdropping, and the pinned fingerprints protect against an attacker who later presents a different certificate. TLS alone does not prove that a peer is honest, which is the job of the protocol checks described next.

### 11.3 Connection handshake

A new connection goes through a short handshake. The handshake begins with a HELLO message. The receiving node answers with a proof-of-work challenge, and the connecting node must find a nonce that satisfies a 19-bit hash puzzle. That takes about half a million hash attempts on average. The exchange then continues through verification and challenge-response messages, the TLS fingerprint is checked against the pinned value, and logic hashes are compared as described in Section 11.5. The handshake ends with an ACCEPT or a REJECT. The puzzle makes it expensive to open large numbers of fake connections.

### 11.4 Burst protection

If more than 30 connection attempts arrive per minute from one IPv4 /24 network, or one IPv6 /48 network, the puzzle difficulty for that network rises by 4 bits. That is sixteen times the work for each attempt from the affected network. The adjustment is automatic.

### 11.5 Logic hashes and trust

Each node computes a SHA-256 digest of its own source files and reports it during the handshake. Peers running identical code produce identical digests. A peer whose digest is neither the node's own nor one of the known-good values in the configuration is treated as low trust. The migration notes describe this check as a performance and compatibility

hint rather than a security gate. In practice, mixed-version networks should be avoided. Operators should upgrade all nodes together or add the new digest to the known-good list.

## 11.6 Peer scoring and bans

Each peer carries a ban score. Invalid messages add 10 points, oversized messages add 50, sync timeouts add 5, and sustained flooding above twice the allowed message rate adds 20. A peer that reaches 100 points is banned. Scores halve every 600 seconds, so an occasional mistake does not follow a peer indefinitely. A separate reputation record persists across restarts and can reflect good conduct over months. Each peer's bandwidth is capped at 10 MB per second.

## 11.7 Peer limits and discovery

A node keeps at most 30 peers and aims to keep at least 3. Discovery uses a Kademlia distributed hash table. Each node has a 160-bit identifier derived from its public key, and peers are organized into buckets by distance. New nodes learn their first peers from DNS seed hostnames, which operators publish as A or AAAA records, and the seeder queries those names every 600 seconds by default. Peers can also exchange peer lists directly.

## 11.8 Reaching peers behind NAT

Many home devices sit behind network address translation and cannot accept incoming connections by default. Visold includes several techniques for this. STUN discovers a device's public address. ICE tests candidate connection paths and ranks them. UPnP asks the local router to open a port. Hole punching lets two peers connect directly through their routers at the same time. A relay helper process carries traffic when no direct path works. These mechanisms are implemented, but the verification environment could not exercise them on real networks, as Section 18 explains.

## 11.9 Message types

The protocol defines 43 message types. They fall into groups. Handshake messages include HELLO, the proof-of-work challenge, VERIFY, ACCEPT and REJECT. Block and chain messages request and deliver blocks, chains, compact blocks, missing transactions and block chunks. Transaction messages carry ordinary and layer 2 transactions. Snapshot messages support fast sync. Consensus messages carry validator votes,

hashrate reports and rate-defection evidence. Discovery messages include ping, pong, and requests and responses for peer lists. Identity and capability messages support name bindings and feature discovery.

## 11.10 Light clients

A simplified payment verification client, or SPV client, can confirm that a transaction is included in a block. It does this by downloading block headers and a Merkle proof rather than full blocks. This suits phones and other constrained devices that only need to verify payments.

## 12. Storage, pruning and fast sync

### 12.1 Storage backends

The default storage backend is SQLite, a file-based database that needs no separate server. Optional adapters exist for RocksDB and PostgreSQL, and Redis can serve as a cache. Operators select the backend with the `VISOLD_DB_BACKEND` setting. The storage layer presents the same interface whichever backend is used, so the rest of the software does not change.

### 12.2 Rolling-window pruning

Full transaction data is kept for the most recent 600 blocks, which is about ten hours at the target block time. Older blocks keep a header record containing the hash, the previous hash, the Merkle root, the state root, the timestamp, the difficulty and the nonce. Headers are stored in a separate table so the chain's structure remains verifiable. A pruned node records a watermark so that peers know it cannot serve older transactions. Pruning runs in a background thread and does not hold up block processing. The feature is on by default and can be adjusted through environment settings.

### 12.3 State snapshots

Every 600 blocks, the node writes a compressed snapshot of its full state. The snapshot covers balances, nonces, contract metadata, and validator roles and stakes. It includes the state root, which must match the root recorded in the block header, so any recipient can verify the snapshot without trusting its sender. The node keeps the three most recent snapshots and removes older ones. Snapshot creation runs in a background thread.

## 12.4 Fast sync

A new node with no blocks can use fast sync. It asks peers for a snapshot manifest, chooses the most recent snapshot, and downloads it in chunks of 1 MB from up to eight peers at once. Each chunk has a 30-second timeout and up to three retries. The node verifies the state root and restores the state, then fetches the blocks that followed the snapshot in the usual way. Fast sync is attempted only when the chain is at least 600 blocks tall. If any step fails, the node falls back to the full sync path, so a failed fast sync does not leave the node stuck.

## 12.5 State pruning

A separate state pruner removes historical state data that is no longer needed, which bounds database growth. The rolling window and the state pruner work together: one trims old block bodies and the other trims old state.

## 12.6 Practical guidance

A node that keeps only recent history cannot answer requests for old transactions. Operators who need a full archive should turn off rolling pruning and budget for the storage it requires. Operators on phones or small servers can rely on pruning and fast sync to stay within their limits.

# 13. Mempool and MEV protection

## 13.1 The mempool

The mempool holds transactions that have been announced but not yet included in a block. It is capped at 5,000 transactions. Entry requires a fee at or above the node's minimum of 1,000 satoshi. Block builders order transactions by fee, with the highest first, then by age, with the oldest first, and then by transaction identifier as a final tie-breaker. Transactions past their expiry are dropped. A node also drops an identity claim whose sender can no longer afford the fee.

## 13.2 What MEV is

Maximal extractable value, usually shortened to MEV, is profit that a block producer or a fast observer can capture by reordering, inserting or delaying transactions. The best-known forms are front-running, where a bot copies a pending trade and places its own

ahead of it, and sandwich attacks, where a bot buys before a victim's trade and sells after it. Such activity takes value from ordinary users even when the chain itself follows its rules.

### 13.3 The commit-reveal scheme

Visold includes an optional commit-reveal scheme. In the commit phase, the sender publishes only a hash that binds the transaction identifier, the sender and a nonce. Nobody can see the amount, the receiver or the memo. After a delay, which is three blocks in the current configuration, the sender reveals the full transaction. Nodes check the revealed transaction against the earlier commitment before accepting it. Because the details stayed hidden until the commitment was already on the chain, outside bots cannot front-run them.

The scheme is off by default. An operator enables it by setting the environment variable `VISOLD_MEV_PROTECT` to 1. RPC methods report the scheme's status and accept commitments. The module's header describes the reveal as taking place in the next block, but the configuration sets a delay of three blocks, and this paper follows the configuration.

### 13.4 What the scheme does not protect against

The project's own documentation is explicit about the limit, and this paper repeats it. The scheme protects against outside bots only. It does not protect against a malicious validator. A validator receives the full revealed transaction before it builds its block proposal, and it can still place a front-running transaction ahead of the revealed one. Full protection against validators would require threshold encryption, so that no single validator can read a transaction until after it has been included. The notes list this as a research-track item tied to a planned ZK-proof integration phase in the first quarter of 2027. Users should understand this limit before relying on the feature.

## 14. Self-healing, resilience and monitoring

### 14.1 Purpose

The Self-Healing Blockchain System, or SHBS, is a monitoring and response layer. It watches the chain for abnormal patterns, classifies what it finds, and responds in graded steps. It uses fixed, documented rules rather than machine-learning models, which keeps

its decisions predictable and auditable. Operators can read its logs, check its risk status, cast votes on rollbacks and lift freezes through RPC methods.

## 14.2 Layers

The system is organized into five layers.

| Layer | Name              | What it does                                                                                                     |
|-------|-------------------|------------------------------------------------------------------------------------------------------------------|
| 1     | Monitoring        | Collects measurements on transactions, gas use, validator activity and reentrancy patterns through a central bus |
| 2     | Anomaly detection | Maintains running baselines and flags values that depart sharply from them                                       |
| 3     | Decision engine   | Assigns each anomaly a severity using fixed rules and confidence thresholds                                      |
| 4     | Healing actions   | Writes logs, applies rate limits, freezes accounts or contracts, and alerts validators                           |
| 5     | Rollback engine   | Keeps snapshots of recent blocks and can reverse a set number of blocks deterministically                        |

## 14.3 Severity levels

The decision engine sorts findings into four levels. The table gives examples and the confidence each level requires.

| Level    | Examples                                                                                                                                                          | Confidence requirement                                      |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| CRITICAL | Proven double-signing; a fund drain above 100,000 VSD with confidence above 0.8; reentrancy in a contract holding more than 50,000 VSD; detected supply inflation | At least 0.8, plus a check on the attacker's expected value |
| HIGH     | A fund drain above 10,000 VSD; a reentrancy pattern; a transaction spike together with a mempool flood; more than half of validators inactive                     | At least 0.6                                                |

| Level  | Examples                                                                                                                                             | Confidence requirement |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| MEDIUM | A transaction-rate spike above 3.5 standard deviations, or a gas spike above 4; a flood from one sender; gas exhaustion; a single inactive validator | Statistical threshold  |
| LOW    | Wash trading; anomalies below the HIGH thresholds; low-confidence suspicion of validator collusion                                                   | Recorded for review    |

The rules are deterministic, so the same inputs always produce the same classification. The confidence requirements stop weak evidence from triggering severe responses.

## 14.4 Healing actions

Each severity level maps to actions. The table lists them and states how far each one reaches.

| Action          | Effect                                                          | Scope             |
|-----------------|-----------------------------------------------------------------|-------------------|
| Log             | Records the event in an immutable audit log                     | Local to the node |
| Rate limit      | Slows transactions from a single address using a sliding window | Local to the node |
| Freeze account  | Blocks an account for 600 seconds                               | Local to the node |
| Freeze contract | Freezes a contract                                              | Local to the node |
| Validator alert | Sends an alert with its evidence to connected validators        | Network message   |

The important scope point is that freezes are local. A frozen account is recorded only by the node that froze it. The freeze is never saved as a binding decision, is not gossiped as a rule, and is not part of consensus. Two honest nodes can therefore disagree about whether an account is frozen. Operators should read freezes as local protective measures, not as protocol decisions.

## 14.5 Rollbacks

The rollback engine keeps snapshots of the state before each recent block, in a ring buffer of limited size. It can reverse a chosen number of blocks by undoing their rewards, transfers and state changes in reverse order. Critical rollbacks are not executed on one operator's word. The system broadcasts a rollback proposal to validators, each validator checks the evidence and signs or rejects the proposal, and the rollback goes ahead only when enough validators approve. Operators can run a dry run first to see what would change. A limit on rollbacks within a rolling window prevents repeated reversals.

## 14.6 Resilience components

Four components sit alongside the self-healing layers. The panic circuit breaker can halt risky operations and can be inspected and reset through RPC. The hardened core adds overlay checks around critical code paths. The safety invariants test that money is conserved and that supply is consistent, and they can be run on demand through the invariant-check method. The sentinel is a standby monitor, described next.

## 14.7 The sentinel

The sentinel is a lightweight background process for validator operators. It checks the health of a primary node through repeated RPC health checks, and it watches whether the primary's chain height keeps advancing. When it decides the primary has failed, it marks itself active and alerts a human operator. The sentinel does not import the validator's signing key, and it does not begin casting votes on its own. The code states plainly that an automatic vote-casting takeover does not exist in this codebase, which corrected an earlier claim. Recovery and key handling therefore remain manual, and any operational plan should reflect that.

## 14.8 Monitoring interfaces

Operators can read node metrics, database sizes, invariant results, circuit-breaker state, sentinel status, SHBS status, SHBS risk status and the SHBS action log through RPC. Section 17 lists the methods.

## 15. Governance and upgrades

### 15.1 Versions

The configuration reports software version 11.0.0.0, and the protocol version constant is 1. The two numbers are separate. A protocol version change alters consensus rules, while a software version change may or may not do so. Some older documentation headers carry a different version label. This paper follows the configuration.

### 15.2 The upgrade lifecycle

A protocol upgrade passes through defined phases, shown in the table.

| Phase     | Meaning                                                                                                         |
|-----------|-----------------------------------------------------------------------------------------------------------------|
| DORMANT   | The proposal is announced, and signaling has not begun                                                          |
| SIGNALING | Miners include the new version in their blocks, and the engine counts the share of recent blocks that signal it |
| LOCKED_IN | At least 75 percent of blocks in the signaling window support the change, and an activation height is fixed     |
| ACTIVE    | The activation height has been reached, and the new rules apply                                                 |
| FAILED    | A rollback or an emergency disable has ended the proposal                                                       |

### 15.3 Tools

Operators can read upgrade status and governance state, propose an upgrade through RPC, and invoke an emergency disable. The rollback mechanism described in Section 14 is the last resort for consensus problems. The governance engine limits how often rollbacks can occur within a window of blocks, so the system cannot be pushed into repeated reversals.

## 15.4 Activation heights

Rules that change how the protocol behaves carry activation heights. Several activation parameters in the current configuration are set to zero, which means those rules apply from genesis. The configuration comments explain that the reward rules were deployed from genesis for this reason. They also warn that changing rules after public launch without an activation mechanism would fork the chain, because nodes running the old rules would reject blocks produced under the new ones. Post-launch changes should therefore be introduced with a future activation height.

## 15.5 Coordinated upgrades

Because each node's logic hash depends on its source code, nodes running different versions treat each other as low trust during the handshake, as Section 11.5 explains. Upgrades should therefore be coordinated so that all participants switch together, after the usual multi-node testing has been completed.

# 16. Wallets, names and roles

## 16.1 Wallets

A wallet holds an encrypted keystore and a set of derived keys. The command line can create a wallet, and the interactive interface guides a new user through creating an account on first run. The wallet signs transactions with a private-key object that is built once and reused, which improves speed. The private key itself is never sent over the network. Only the public key and the signature accompany a transaction.

## 16.2 Names

The identity system is a decentralized name service. A user claims a name through a register transaction, which pays the minimum identity-claim fee of 1,000 satoshi. The name is recorded against the wallet address that claimed it, so the network can show which address holds a given name. Name claims are relayed between nodes, and a node drops a pending claim from its mempool if the sender can no longer pay the fee.

## 16.3 Roles

The role system records each participant's standing, and three roles are defined. A miner registers with at least 10 VSD of stake. An investor registers with at least 200 VSD of stake. A high transactor is a qualifier rather than a registration. The storage layer tracks

the cumulative volume of each address, and addresses whose volume reaches 1,000 VSD are flagged as high transactors. Unstaking is also carried out through a register transaction.

Two points are worth stating clearly. First, the code documents the high transactor category alongside the other roles, but the consensus effects of that flag are not described in the modules reviewed for this paper. Second, validator votes are weighted by the validator set and stake amounts held in storage. The archive does not show a direct link from the investor role to validator registration, and that relationship should be documented before the network launches.

## 16.4 Accounts in the node

The node has a user-account layer. On first run, the interactive interface creates an account and then starts the node. The account layer shares a module with the node's main class, because the two are tightly connected.

# 17. Interfaces: command line, terminal and RPC

## 17.1 Command line

Running the entry file with the cli option opens command-line tools. Subcommands cover wallet creation and other node tasks, and the help option lists them. The command-line tools were tested in the verification process described in Section 18, including wallet creation.

## 17.2 Interactive terminal

Running the entry file with no options starts the interactive node. The terminal interface draws menu panels and walks a new user through account creation, then starts the full node services. The verification process ran scripted sessions of this startup flow.

## 17.3 JSON-RPC server

The node runs a JSON-RPC 2.0 server over HTTP for wallets and block explorers. The server listens on localhost only, on the node's P2P port plus one, which is 8339 by default. Every request must present a bearer token, a random 256-bit value generated on first run. Because the server accepts connections only from the local machine, remote access requires the operator to set up a secure tunnel or proxy, and that should be done with care.

## 17.4 Method catalog

The server exposes 59 methods, grouped in the table below.

| Group                         | Count | Examples                                                                                                                |
|-------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------|
| Chain and transaction queries | 11    | getbalance, getblock, getblockheader, getmerkleproof, sendtransaction, getmempool                                       |
| Network and peers             | 5     | getpeerinfo, findpeerswith, getpeerreputation, getcapabilities, getsentinelstatus                                       |
| Governance and protocol       | 7     | getprotocolversion, getchaininfo, getupgradestatus, proposeupgrade, emergencydisable, forkchoice                        |
| Diagnostics and safety        | 7     | getmetrics, checkinvariants, getdbsizes, getcircuitbreaker, resetcircuitbreaker, getstatepruneinfo                      |
| MEV protection                | 2     | mevcommit, getmevstatus                                                                                                 |
| Slashing evidence             | 1     | submitslashingevidence                                                                                                  |
| Smart contracts               | 11    | deploycontract, callcontract, simulatecall, estimategas, getcontract, getcontractevents, analyzecontract                |
| Self-healing controls         | 9     | shbs_status, shbs_log, shbs_vote, shbs_unfreeze, shbs_rollback_dry_run                                                  |
| Layer 2 operations            | 6     | vsd_getBalance, vsd_getL2StateRoot, vsd_sendL2Transaction, vsd_depositToL2, vsd_withdrawFromL2, vsd_triggerManualRollup |

Taken together, the catalog covers ledger queries, network status, governance, diagnostics, contracts, self-healing controls and layer 2 operations, so the interface reaches every major part of the node.

## 18. Testing and verification

### 18.1 Equivalence of the modular build

The move from the single file to the package was made without changing behavior, and the project's migration log records how this was checked. Each moved statement was copied by a script rather than retyped by hand. The checks found no lost or unexpected lines in the moved code, no import cycles among the 117 modules, and identical instruction streams for 1,417 functions and methods. Of 2,877 namespace records compared, 2,873 matched exactly. The four differences were intentional: two changes to how the source identity is computed, one change to a block download import, and a start-time counter. Behavioral comparison of the original and modular builds produced identical output, exit codes and files in eight scenarios. These covered the version command, the embedded self-tests, the command-line help, wallet creation and two scripted node start-ups.

### 18.2 Test suites

The embedded suite runs with the test option. The migration log records 138 of 138 embedded tests passing in both the original and the modular builds. Separately, the tests directory contains 18 files with 122 test functions. Their names show what they cover: rollback regressions, UDP security and peer recovery, consensus hardening, crypto acceleration, sync import optimization, virtual machine audit findings, and layer 2 handling of failed batches. Several of these files were added in October 2026 to pin down specific fixes.

### 18.3 Recent fixes

The archive contains reports for several rounds of fixes in early October 2026. At a high level, they address four areas. In consensus determinism, non-finite difficulty and timestamps are rejected, block size validation uses the fixed consensus cap, and unknown transaction types are refused rather than accepted by default. In state accounting, a stake released and re-registered within the same block is no longer counted twice, nested contract value transfers settle only when the child call succeeds, typed-storage metadata persists correctly, and address encoding inside the virtual machine round-trips correctly. In signatures and performance, transaction signing uses a domain-separated encoding, and signature work is accelerated without changing transaction formats or consensus rules. In peer handling, recovery over UDP and the speed of sync import were improved.

## 18.4 What the verification did not cover

The verification environment had no network access, no GPU and none of the optional database backends. As a result, real multi-node behavior has not been exercised. NAT traversal, UPnP, STUN and the relay helper were not run on real networks. Long mining runs, chain reorganizations under real latency and the optional backends were verified only structurally, meaning that the code was checked for consistency rather than run under production-like conditions. The project's own guidance is to run a multi-node test before replacing an existing node. This paper treats that test as a precondition for any public network.

## 18.5 Verification tools

The tools directory holds the checks used for the migration and for ongoing work. An architecture checker must pass after every edit. A static verifier compares structure and import behavior with the original file. A runtime verifier compares bytecode and name resolution. An import-order tester checks that modules load correctly in any order. A behavior runner compares the original and modular builds side by side. A rebuild script re-verifies each stage of the migration, and a generator recreates the package from the original file. Together these tools make the equivalence claims reproducible by anyone who has the original file.

# 19. Security model

## 19.1 Who and what is protected

The security model considers ordinary users, miners, validators, node operators, layer 2 sequencers, and attackers who may try to forge or double-spend coins, censor transactions, front-run trades, flood the network or exploit contracts. The assets at stake are VSD balances, contract funds, channel deposits, layer 2 balances and the integrity of the ledger itself.

## 19.2 Threats and mitigations

The table sets out the main threats, the defenses in the code, and the concerns that remain.

| Threat                                | Mitigation in the code                                                         | Remaining concern                                        |
|---------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------|
| Large numbers of fake peers           | Proof-of-work puzzle per connection, burst detection, peer scoring             | Honest peers also pay a small work cost                  |
| Validators signing conflicting blocks | Double-sign evidence, independent verification, slashing                       | Depends on evidence reaching nodes                       |
| Manipulated difficulty or timestamps  | Bounded adjustments, median time past, future-drift limit, finite-value checks | Bounds limit timing games but do not eliminate them      |
| Miners exceeding hashrate caps        | Local cap plus consensus-level audit                                           | Automatic slashing is off by default                     |
| Malformed or oversized messages       | Size limits, invalid-message scoring and bans                                  | Depends on the limits being set correctly                |
| Mempool flooding                      | Capacity cap, minimum fee, expiry                                              | Pressure remains possible during bursts                  |
| Front-running by outside bots         | Optional commit-reveal scheme                                                  | Off by default                                           |
| Front-running by validators           | Not mitigated                                                                  | Needs threshold encryption, which is still research      |
| Contract exploits                     | Sandbox, static analysis, typed values, time-locks, staged transfers           | Pattern-based analysis, and no third-party VM audit      |
| Key theft                             | Encrypted keystore, Argon2id option, offline backup phrase                     | No recovery if the phrase is lost                        |
| Fraudulent layer 2 batches            | Proof interface and mainnet guard                                              | No production proof backend configured by default        |
| Chain-wide anomalies                  | Self-healing system, circuit breaker, invariant checks                         | Freezes are local, and rollbacks need validator approval |
| Mixed software versions               | Logic-hash trust levels                                                        | Requires coordinated upgrades                            |

| Threat                      | Mitigation in the code                    | Remaining concern           |
|-----------------------------|-------------------------------------------|-----------------------------|
| Errors in supply accounting | Conservation checks and an inflation rule | Supply has no cap by design |

### 19.3 Reading the table

Some of these defenses are implemented and covered by tests. Some are configured but switched off by default, and some are research plans. The table does not rank them by strength. Operators should check which protections are active in their own configuration before relying on them.

### 19.4 Reporting security issues

The archive does not include a security contact or a disclosure policy. Before public launch, maintainers should publish both, so that researchers know where to send findings and what timeline to expect before any public disclosure.

### 19.5 Audit status

The archive includes internal audit findings and their fixes, dated as recently as October 2026. It does not include an independent third-party security audit of the consensus, networking, virtual machine or cryptographic code. Anyone planning to hold significant value on the network should commission one.

## 20. Status, roadmap and known limitations

### 20.1 Where the project stands

The codebase is large, internally consistent and well documented. Many features are implemented and covered by unit-level and structural checks. The project's own configuration describes the current build as a development build, in which every node operator wipes the database before running. The consensus rules are fixed from genesis. Nothing in the archive shows the software running as a live multi-node network, and the verification environment could not exercise the network layer. The software has therefore not yet been demonstrated in a live multi-node setting.

## 20.2 Planned work named in the project notes

The project notes mention several next steps. They place a ZK-proof integration phase in the first quarter of 2027. The notes do not spell out its full scope, but the proof backend interface suggests it would supply the validity proofs that the layer 2 rollup needs. The notes also describe research into threshold encryption, which would give validator-level MEV resistance. Splitting the largest classes into smaller parts is described as a riskier follow-up. Automatic rate-defection slashing is to be enabled only after several weeks of observation and a coordinated restart. An activation-height mechanism is recommended for changing rules after public launch. Finally, a multi-node test campaign is a stated prerequisite for replacing the original file. These are the project's own intentions, not commitments made in this paper.

## 20.3 Known limitations

The table summarizes the main limitations and why each one matters.

| Area          | Limitation                                                    | Why it matters                                        |
|---------------|---------------------------------------------------------------|-------------------------------------------------------|
| Supply        | No cap; 0.1 VSD per block after about 17.5 years              | Long-run supply grows every year                      |
| Finality      | With three equal-stake validators, all three votes are needed | One offline validator stops BFT finality              |
| Layer 2       | No proof backend configured by default                        | Batches are not proven valid in the default setup     |
| MEV           | Commit-reveal is off by default and does not stop validators  | Front-running by validators remains possible          |
| Hashrate      | Automatic slashing for throttling is off by default           | Defection is observed but not penalized until enabled |
| Sentinel      | Alerts a human and does not take over signing                 | Failover depends on operator response                 |
| Freezes       | Local to each node                                            | Nodes can disagree about frozen accounts              |
| Backup phrase | Word list is padded or trimmed                                | Phrases may not restore in other BIP-39 wallets       |

| Area         | Limitation                                                                          | Why it matters                             |
|--------------|-------------------------------------------------------------------------------------|--------------------------------------------|
| Rule changes | Rules fixed from genesis; activation mechanism not yet used for post-launch changes | Changes need careful, coordinated planning |
| Database     | Development builds expect fresh databases                                           | Not yet a migration-ready release          |
| Testing      | Real networking, NAT traversal, GPU and optional databases unexercised              | Real-world behavior is unproven            |

## 20.4 What a reader should check first

Anyone evaluating Visold should start with the supply schedule in Section 8, the finality rules in Section 7, the status notes in this section and the security table in Section 19. They should then run the test suite, set up a small multi-node network under realistic conditions, and review the audit findings directly.

## 21. Disclaimer

This paper describes software, its design and its present state. It is not an offer or solicitation to buy, sell or hold any asset, and it makes no forecast of price, returns or adoption. Nothing in it is legal, financial or tax advice. Readers should review the source code themselves, run their own tests and consult qualified advisers before relying on the software or holding VSD. The figures in this paper come from the archive as it stood on October 8, 2026, and will change as the code changes.

## Appendix A. Parameter reference

The table lists the main configured values in the archive, with the section of this paper that explains each one.

| Parameter     | Value                            | Section |
|---------------|----------------------------------|---------|
| Coin symbol   | VSD                              | 3       |
| Smallest unit | 1 VSD equals 100,000,000 satoshi | 3       |
| Chain ID      | vsd-mainnet-1                    | 4       |

| Parameter                                                                | Value                                                   | Section |
|--------------------------------------------------------------------------|---------------------------------------------------------|---------|
| Target block time                                                        | 60 seconds, with a tolerance of plus or minus 5 seconds | 7       |
| Initial difficulty                                                       | 5.15, bounded between 4.0 and 63.0                      | 7       |
| Difficulty averaging window                                              | 20 blocks, linearly weighted                            | 7       |
| Solve-time clamp per block                                               | 1 to 360 seconds                                        | 7       |
| Maximum change per difficulty adjustment                                 | Factor of 1.5                                           | 7       |
| Bootstrap blocks at initial difficulty                                   | 3                                                       | 7       |
| Median time past window                                                  | 11 blocks                                               | 6       |
| Maximum future timestamp drift                                           | 7,200 seconds                                           | 6       |
| Consensus block size cap                                                 | 4,096,000 bytes                                         | 6       |
| Candidate block size floor                                               | 1,048,576 bytes, or 1 MiB                               | 6       |
| Candidate size adjustment                                                | Up 10 percent, down 5 percent                           | 6       |
| Initial block reward                                                     | 10 VSD                                                  | 8       |
| Minimum block reward                                                     | 0.1 VSD                                                 | 8       |
| Reward decay                                                             | Multiplied by 0.99 every 20,000 blocks                  | 8       |
| Proposer share                                                           | 20 percent                                              | 8       |
| Active miner share                                                       | 45 percent                                              | 8       |
| Validator share                                                          | 35 percent                                              | 8       |
| Miner activity window                                                    | 100 blocks                                              | 8       |
| Transfer fee                                                             | 1 percent of the amount                                 | 8       |
| Minimum fee for identity claims, certain registrations and mempool entry | 1,000 satoshi                                           | 8       |
| Reward concentration alert                                               | Above 40 percent to one address                         | 8       |
| Minimum miner stake                                                      | 10 VSD                                                  | 8       |

| Parameter                                   | Value                                                           | Section  |
|---------------------------------------------|-----------------------------------------------------------------|----------|
| Minimum investor stake                      | 200 VSD                                                         | 8        |
| High transactor threshold                   | 1,000 VSD of tracked volume                                     | 16       |
| BFT finality threshold                      | 66.67 percent of total stake                                    | 7        |
| Minimum validators for BFT mode             | 3                                                               | 7        |
| Proof-of-work finality depth                | 6 blocks, or 20 during bootstrap                                | 7        |
| Slashing rate for malicious votes           | 10 percent of stake                                             | 7        |
| Rate audit window                           | 500 blocks                                                      | 7        |
| Rate audit cadence                          | Every 50 blocks                                                 | 7        |
| Rate audit flag ratio                       | Above 2.0                                                       | 7        |
| Consecutive flagged windows before slashing | 3                                                               | 7        |
| Minimum audited hashrate share              | 3 percent                                                       | 7        |
| Offence counter halving interval            | 10,000 blocks                                                   | 7        |
| Fork signaling window                       | 100 blocks                                                      | 7 and 15 |
| Fork signaling threshold                    | 75 percent                                                      | 7 and 15 |
| Default P2P port                            | 8338                                                            | 11       |
| Default RPC port                            | 8339                                                            | 17       |
| Maximum and minimum peers                   | 30 and 3                                                        | 11       |
| Connection puzzle difficulty                | 19 bits                                                         | 11       |
| Burst threshold                             | More than 30 connections per minute from one /24 or /48 network | 11       |
| Burst difficulty increase                   | 4 bits                                                          | 11       |
| Peer ban threshold                          | 100 points                                                      | 11       |
| Ban score decay interval                    | 600 seconds                                                     | 11       |

| Parameter                      | Value                      | Section |
|--------------------------------|----------------------------|---------|
| Peer bandwidth cap             | 10 MB per second           | 11      |
| DNS seed query interval        | 600 seconds                | 11      |
| TLS certificate validity       | 90 days                    | 5       |
| Mempool capacity               | 5,000 transactions         | 13      |
| Default transaction expiry     | 1 hour                     | 6       |
| Maximum transaction expiry     | 24 hours                   | 6       |
| Commit-reveal delay            | 3 blocks                   | 13      |
| Layer 2 batch size             | 2,048 transactions         | 10      |
| Layer 2 batch maximum age      | 30 seconds                 | 10      |
| Layer 2 state roots retained   | 128                        | 10      |
| Freeze duration                | 600 seconds                | 14      |
| Channel dispute timeout range  | 10 to 50,000 blocks        | 9       |
| Rolling pruning window         | 600 blocks                 | 12      |
| Snapshot interval              | Every 600 blocks           | 12      |
| Snapshots retained             | 3                          | 12      |
| Fast sync minimum chain height | 600 blocks                 | 12      |
| Snapshot chunk size            | 1 MB                       | 12      |
| Parallel sync peers            | 8                          | 12      |
| Chunk timeout and retries      | 30 seconds, with 3 retries | 12      |

## Appendix B. Glossary

| Term    | Meaning                                                                    |
|---------|----------------------------------------------------------------------------|
| Address | The identifier that receives and sends VSD, beginning with the letters VSD |

| Term                 | Meaning                                                                               |
|----------------------|---------------------------------------------------------------------------------------|
| Base58               | A text encoding that avoids easily confused characters, used for addresses            |
| BFT                  | Byzantine fault tolerance: agreement that holds even when some participants misbehave |
| Bootstrap            | The early phase of a chain, when few validators exist and default rules apply         |
| Bounded context      | A group of code responsible for one area, such as storage or networking               |
| Block                | A batch of transactions with a header that links it to the previous block             |
| Block reward         | New VSD paid to block producers and participants for each block                       |
| BIP-39               | A widely used standard for word-based backup phrases                                  |
| Coinbase transaction | The first transaction in a block, which pays the block reward                         |
| Commit-reveal        | A scheme that hides transaction details first and reveals them later                  |
| Consensus            | The process by which nodes agree on which blocks are valid and in what order          |
| Conservation check   | A test confirming that balances add up to total issuance                              |
| Double-signing       | Signing two different blocks at the same height, which is an offence                  |
| ECDSA                | The signature scheme used for transactions                                            |
| Finality             | The point at which a block is treated as effectively irreversible                     |
| Fork                 | A split in the chain caused by nodes following different rules or blocks              |

| Term           | Meaning                                                                                 |
|----------------|-----------------------------------------------------------------------------------------|
| Freeze         | A local, temporary block on an account or contract, applied by one node                 |
| Gas            | A unit that measures the work a contract performs and sets its cost                     |
| Genesis block  | The first block of the chain, from which all others follow                              |
| Hashrate       | The rate at which a miner computes hashes, a measure of mining power                    |
| HD wallet      | A wallet whose keys all derive from one secret                                          |
| Kademlia       | A distributed hash table used to find peers                                             |
| Layer 2        | A system that processes transactions off the main chain and commits results back to it  |
| Logic hash     | A digest of a node's source code, used to judge whether peers run compatible software   |
| Mempool        | A node's pool of unconfirmed transactions                                               |
| Merkle root    | A single hash that summarizes every item in a list, such as the transactions in a block |
| MEV            | Profit captured by reordering or delaying transactions                                  |
| NAT traversal  | Techniques that let devices behind routers accept or make connections                   |
| Nonce          | A number that miners vary while searching for a valid block hash                        |
| Proof of stake | A system in which voting power comes from locked-up stake                               |
| Proof of work  | A system in which blocks are secured by computational effort                            |
| Rollup         | A layer 2 design that posts batch summaries to the main chain                           |

| Term       | Meaning                                                                            |
|------------|------------------------------------------------------------------------------------|
| Satoshi    | The smallest unit of VSD, equal to one hundred-millionth of a coin                 |
| Sentinel   | A standby monitor that watches a primary validator node and alerts operators       |
| Slashing   | Removal of part of a validator's stake for misbehavior                             |
| Snapshot   | A compressed copy of chain state at a given height, used for fast sync             |
| SPV        | Simplified payment verification, a light-client method based on headers and proofs |
| Stake      | VSD locked by a validator or participant to back its role                          |
| State root | A commitment to the full state of balances and data after a block                  |
| Validator  | A participant that locks stake and votes on blocks                                 |
| VRF        | Verifiable random function: an output that anyone can check against a public key   |
| VVM        | The Visold Virtual Machine, the engine that runs smart contracts                   |

## Appendix C. Where to find each topic in the code

The paths below are relative to the root of the archive. They let a reader check any claim in this paper against the source.

| Topic                       | Location                                                |
|-----------------------------|---------------------------------------------------------|
| Constants and configuration | visold/kernel/config.py                                 |
| Block reward calculation    | visold/chain/blockchain.py, function compute_reward_sat |
| Reward distribution rules   | visold/economics/rewards.py                             |

| Topic                                 | Location                                                                   |
|---------------------------------------|----------------------------------------------------------------------------|
| Transaction fee calculation           | visold/ledger/transaction.py, function compute_fee_sat                     |
| Transaction and block structures      | visold/ledger/transaction.py and visold/ledger/block.py                    |
| Difficulty adjustment                 | visold/consensus/difficulty.py                                             |
| BFT threshold check                   | visold/kernel/units.py, function bft_threshold_met                         |
| Consensus engine and validator voting | visold/chain/consensus_engine.py                                           |
| Roles and staking                     | visold/chain/roles.py                                                      |
| Slashing evidence                     | visold/consensus/slashing.py                                               |
| Rate-defection audit                  | visold/consensus/rate_defection.py                                         |
| Hashrate governor                     | visold/consensus/hashrate_governor.py                                      |
| Mining safety guard                   | visold/consensus/mining_safety.py                                          |
| Governance and upgrades               | visold/governance/engine.py                                                |
| Signatures and addresses              | visold/crypto/ecc.py                                                       |
| Verifiable random function            | visold/crypto/vrf.py                                                       |
| Keystore and backup phrase            | visold/crypto/keystore.py and visold/crypto/mnemonic.py                    |
| Wallet and key derivation             | visold/wallet/                                                             |
| Virtual machine                       | visold/vm/engine.py, visold/vm/opcodes.py and visold/vm/static_analyzer.py |
| Layer 2 state and proofs              | visold/rollup/                                                             |
| Layer 2 sequencer                     | visold/chain/sequencer.py                                                  |
| Peer-to-peer networking               | visold/network/p2p.py and visold/network/udp/                              |
| NAT traversal                         | visold/network/nat/                                                        |
| TLS and reputation                    | visold/network/tls.py and visold/network/reputation.py                     |

| Topic                          | Location                                                       |
|--------------------------------|----------------------------------------------------------------|
| Discovery                      | visold/network/kademlia.py and<br>visold/network/dns_seeder.py |
| Light client                   | visold/network/spv.py                                          |
| Storage and pruning            | visold/storage/                                                |
| Mempool and MEV protection     | visold/mempool/                                                |
| Self-healing system            | visold/selfhealing/                                            |
| Resilience and sentinel        | visold/resilience/                                             |
| JSON-RPC server                | visold/api/rpc_server.py                                       |
| Node composition               | visold/node/visold_node.py                                     |
| Name service                   | visold/identity/names.py                                       |
| Command line and terminal      | visold/cli/                                                    |
| Embedded tests and test folder | visold/testing/ and tests/                                     |